

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

Мэдээлэл, Холбооны Технологийн Сургууль



БИЕ ДААЛТЫН АЖЛЫН ТАЙЛАН

Параллел ба хуваарилагдсан тооцолол(F.CSM306)
2025-2026 оны хичээлийн жилийн хаврын улирал

Хичээл заасан багш:

Г. Ганбат

Гүйцэтгэсэн:

Г. Эрдэнэболор, В231910014

Б. Өсөхбаяр В231910012

М. Хүслэн В231910030

Л. Батзориг В231910001

Улаанбаатар хот
2026 он

Contents

Хураангуй	3
1 Оршил	3
2 Онолын ойлголт	3
2.1 Merge Sort Алгоритм	3
2.2 Parallel Merge асуудал	4
2.3 Co-rank Алгоритм ба Параллель Нэгтгэл	4
2.4 Ашигласан параллел програмчлалын аргууд	5
2.5 Параллел тооцооллын онолын суурь	5
2.6 Amdahl-ын хууль ба speedup-ийн хязгаар	5
2.7 CPU ба GPU Архитектурын Харьцуулалт	6
2.8 CUDA programming model	6
2.9 Memory hierarchy ба host-device дамжуулалт	7
2.10 OpenMP runtime model	7
2.11 Гүйцэтгэлийн үнэлгээний метрикүүд	7
2.12 Merge үйлдлийг параллелчлэх онолын сорилт	8
3 Туршилтын орчин	8
3.1 Тоног төхөөрөмж (Hardware)	8
3.2 Програм хангамж (Software)	9
4 Туршилтын аргачлал	9
5 Алгоритмын зохиомж	10
5.1 Ерөнхий бүтэц	10
5.2 Өгөгдөл үүсгэх	10
5.3 Sequential хэрэгжилт	10
5.4 Parallel хэрэгжилт (std::thread)	10
5.5 Co-rank ашигласан merge	11
5.6 OpenMP хэрэгжилт	11
5.7 CUDA хэрэгжилт	11
5.8 Нийт ажиллагаа	12
6 Үр дүн ба харьцуулсан шинжилгээ	12
6.1 Тоон үр дүн	12
6.2 CUDA-ийн хугацааны задлал	12
6.3 Execution Time	13
6.4 Speedup	14
6.5 Achievable Performance	15
6.6 CUDA implementation-ийн хязгаарлалт ба тайлбар	15
6.7 Ерөнхий ажиглалт	16
7 Дүгнэлт	16
8 Эх сурвалж	17
Эх сурвалж	17

Хураангуй

Энэхүү бие даалтын ажлаар жагсаалт дээрх нэгтгэх эрэмбэлэлт буюу Merge Sort алгоритмыг дараалсан болон параллель хувилбаруудаар хэрэгжүүлж, гүйцэтгэлийг харьцуулсан болно. Туршилтад 10,000, 100,000, 1,000,000 хэмжээтэй өгөгдөл ашигласан.

Энэхүү харьцуулалтад serial, std::thread, OpenMP болон CUDA хувилбаруудыг авч үзсэн. Өгөгдлийн хэмжээ өсөх тусам параллел хувилбарууд илүү үр ашигтай ажиллаж байгааг үр дүнгээс харах боломжтой. Тухайлбал, OpenMP хувилбар нь 1M элементэд $8.19\times$ хурдатгал үзүүлсэн бол CUDA нь $3.93\times$ хурдатгал үзүүлсэн. Жижиг хэмжээтэй өгөгдөл (10,000) дээр CUDA нь GPU-ийн initialization warm-up хийсний дараа $2.06\times$ хурдатгалтай бусад аргуудыг гүйцэлдсэн.

1 Оршил

Эрэмбэлэлтийн алгоритм нь компьютерийн шинжлэх ухаанд өргөн хэрэглэгддэг үндсэн алгоритмуудын нэг юм. Хэрэв өгөгдлийн хэмжээ том байвал энгийн дараалсан алгоритм удаан ажиллах ба олон цөмт CPU болон параллел програмчлал ашиглан гүйцэтгэлийг сайжруулах шаардлагатай болдог.

Энэхүү ажлын зорилго нь Merge Sort алгоритмыг хэд хэдэн хувилбараар хэрэгжүүлж, тэдгээрийн ажиллах хугацаа болон хурдатгалыг харьцуулах юм.

2 Онолын ойлголт

2.1 Merge Sort Алгоритм

Merge Sort нь **Divide and Conquer** буюу “Хуваагаад эзэл” зарчим дээр суурилсан үр ашигтай эрэмбэлэлтийн алгоритм юм. Энэхүү алгоритм нь өгөгдлийг хамгийн бага нэгж (ганц элемент) болтол нь рекурсив байдлаар хоёр дэд хэсэгт хувааж, дараа нь эрэмбэлэгдсэн байдлаар буцаан нэгтгэх замаар ажилладаг.

Хугацааны нарийн төвөгтэй байдал (Time Complexity)

Алгоритмын ажиллагааны хугацааг дараах рекуррент хамаарлаар тодорхойлно:

$$T(n) = 2T(n/2) + O(n)$$

Энэхүү тэгшитгэлийн бүрэлдэхүүн хэсгүүдийг тайлбарлавал:

- $T(n)$: n хэмжээтэй өгөгдлийг бүрэн эрэмбэлж дуусахад зарцуулах нийт хугацаа.
- $2T(n/2)$: Өгөгдлийг хоёр тэнцүү хэсэгт хувааж, хэсэг бүрийг рекурсивээр эрэмбэлэхэд шаардагдах хугацаа.
- $O(n)$: Хоёр эрэмбэлэгдсэн дэд жагсаалтыг харьцуулалт хийж, нэгтгэх үйл явцад зарцуулагдах шугаман хугацаа.

Master Theorem-ын дагуу дээрх рекуррент хамаарлыг шийдвэл Merge Sort-ийн хугацааны нарийн төвөгтэй байдал нь хамгийн муу болон дундаж тохиолдолд:

$$T(n) = O(n \log n)$$

Алгоритмын алхам (Pseudo-code)

Merge Sort-ийн үндсэн логикийг дараах байдлаар товч илэрхийлж болно:

```
MERGE_SORT(A, p, r):  
    if p < r:  
        q = (p + r) / 2      // Дундаж цэгийг олох  
        MERGE_SORT(A, p, q)  // Зүүн талыг эрэмбэлэх  
        MERGE_SORT(A, q + 1, r) // Баруун талыг эрэмбэлэх  
        MERGE(A, p, q, r)    // Эрэмбэлэгдсэн хэсгүүдийг нэгтгэх
```

- **Давуу тал:** Тогтвортой эрэмбэлэлт хийдэг бөгөөд их хэмжээний өгөгдөл дээр тогтмол $O(n \log n)$ хугацаанд ажилладаг.
- **Сул тал:** Нэгтгэх үйлдлийг гүйцэтгэхэд нэмэлт санах ой ($O(n)$) шаарддаг.

2.2 Parallel Merge асуудал

Merge Sort алгоритмыг параллель аргаар программчлахад хамгийн төвөгтэй байдаг. Учир нь хоёр эрэмбэлэгдсэн массивыг нэгтгэх үед тус оролтын утгын зүүн болон баруун жагсаалтуудын элементүүд, мөн үр дүн болох нэгтгэж гаргах жагсаалтын элементүүд хоорондоо хамааралтай байдаг.

Иймээс merge үйлдлийг шууд олон thread дээр хуваах боломжгүй бөгөөд тусгай арга шаардлагатай.

2.3 Co-rank Алгоритм ба Параллель Нэгтгэл

Параллель тооцоололд хоёр эрэмбэлэгдсэн жагсаалтыг нэгтгэхдээ ажлын ачааллыг олон урсгалд жигд хуваарилах нь чухал байдаг. **Co-rank** алгоритм нь энэхүү асуудлыг шийдвэрлэх гол механизм бөгөөд нийт гаралтын жагсаалтын дурын k индекс бүрт харгалзах оролтын жагсаалтуудын хуваагдах цэгийг (pivot) бие даан олох боломжийг олгодог.

Гаралтын жагсаалтын k -р индекс дэх элементийг тодорхойлохын тулд зүүн жагсаалтын i болон баруун жагсаалтын j индексийг дараах шугаман хамаарлаар олдог:

$$i + j = k$$

Co-rank алгоритм нь хоёртын хайлт ашиглан $O(\log(\min(m, n)))$ хугацаанд дараах логик нөхцөлийг хангах хамгийн оновчтой i индексийг олдог. Үүнд A нь зүүн, B нь баруун жагсаалт гэвэл:

$$(A[i - 1] \leq B[j]) \quad \text{ба} \quad (B[j - 1] \leq A[i]) \quad (1)$$

Энэхүү нөхцөл биелсэн тохиолдолд A жагсаалтын эхний i элемент болон B жагсаалтын эхний j элементүүд нь нэгтгэсэн жагсаалтын эхний k элементийг бүрдүүлэх нь баталгааждаг.

Co-rank-ийн давуу тал

- **Бие даасан байдал (Independence):** Урсгал бүр өөрт оногдсон k индексийн мужийг бусад урсгалаас хамааралгүйгээр (independent) тодорхойлж, нэгтгэх үйлдлийг гүйцэтгэнэ.
- **Синхрончлолын зардал бага (Low Synchronization):** Өгөгдлийн мужууд нь давхцахгүй тул урсгалуудын хооронд *lock* эсвэл бусад төрлийн синхрончлол шаардлагагүй.
- **Ачааллын тэнцвэржүүлэлт (Load Balancing):** Гаралтын массивыг урсгалуудад тэнцүү хэмжээгээр хувааж өгөх боломжтой тул тооцооллын нөөцийг дээд зэргээр ашигладаг.

Псевдо код (Co-rank)

```
function CO_RANK(k, A, m, B, n):
    i_low = max(0, k - n)
    i_high = min(k, m)
    while true:
        i = (i_low + i_high) / 2
        j = k - i
        if i > 0 and j < n and A[i-1] > B[j]:
            i_high = i - 1
        else if j > 0 and i < m and B[j-1] > A[i]:
            i_low = i + 1
        else:
            return i
```

2.4 Ашигласан параллел програмчлалын аргууд

- **std::thread** – CPU дээр олон thread ашигласан
- **OpenMP** – directive ашиглан параллел болгосон
- **CUDA** – GPU дээр co-rank merge хийсэн

2.5 Параллел тооцооллын онолын суурь

Параллел тооцоолол нь нэг асуудлыг хэд хэдэн дэд асуудалд хувааж тэдгээрийг зэрэгцүүлэн (concurrently) гүйцэтгэх замаар нийт ажиллах хугацааг багасгах зорилготой. Параллелчлэлийн хоёр үндсэн загвар:

- **Data parallelism** — нэг үйлдлийг олон өгөгдлөн дээр зэрэгцүүлэн гүйцэтгэх (SIMD/SIMT). GPU-ийн үндсэн загвар.
- **Task parallelism** — өөр өөр үйлдлүүдийг өөр өөр процессор дээр зэрэгцүүлэн гүйцэтгэх (MIMD). CPU-ийн multi-threading-ийн загвар.

Merge sort нь **холимог загвартай**: split болон merge үйлдлүүд нь task parallelism-ээр (зүүн/баруун хэсгүүд тус тусдаа), харин merge доторх элемент бүхий ажиллагаа нь data parallelism-ээр (co-rank-ийн тусламжтайгаар) гүйцэтгэгдэх боломжтой.

2.6 Amdahl-ын хууль ба speedup-ийн хязгаар

Параллел програмын хурдатгал (speedup) нь serial хэсгийн хувиар хязгаарлагдана. Хэрэв програмын f хэсэг параллелчлэхгүй (serial), үлдсэн $1-f$ хэсэг P дахин хурдсаж параллелчилэгдэх боломжтой бол:

$$\text{Speedup}(P) = \frac{1}{f + \frac{1-f}{P}}$$

Хязгаарын утга: $P \rightarrow \infty$ үед $\text{Speedup} \rightarrow \frac{1}{f}$.

Жишээ: Merge sort-ийн дотор:

- Split (recursive) — параллелчлэх боломжтой
- Merge — co-rank ашиглавал параллелчлэх боломжтой
- Memory allocation, sync — **serial хэсэг** ($f \approx 5\% - 10\%$)

Иймд $P = 20$ үед хязгаар: $\text{Speedup} \leq \frac{1}{0.1} = 10\times$. Манай туршилтад OpenMP-ийн speedup $\sim 8\times$ нь онолын хязгаарт ойртсон сайн үр дүн юм.

2.7 CPU ба GPU Архитектурын Харьцуулалт

Орчин үеийн тооцооллын системд CPU болон GPU нь өөр өөр зориулалттай архитектураар зохион байгуулагдсан байдаг. Доорх хүснэгтэд **Intel Core i9-12900H** (Alder Lake архитектур) болон **NVIDIA GeForce RTX 3050 Ti** (Ampere архитектур) загваруудын техникийн үзүүлэлтийг харьцуулан харуулав.

Table 1: Intel i9-12900H ба NVIDIA RTX 3050 Ti архитектурын харьцуулалт

Шинж чанар	CPU (i9-12900H)	GPU (RTX 3050 Ti)
Цөмийн тоо	14 (6P + 8E)	2560 CUDA Cores
Хамгийн өндөр давтамж	5.00 GHz (Max Turbo)	1.48 GHz (Boost Clock)
L3 / L2 Кэш	24 MB (Smart Cache)	2 MB L2 / 80 KB per SM
Branch Prediction	Маш нарийн төвөгтэй	Хязгаарлагдмал (SIMT)
Latency (Хоцрогдол)	Маш бага (Low Latency)	Өндөр (High Latency)
Throughput (Нэвтрүүлэх чадвар)	Дунд (Serial optimized)	Маш өндөр (Parallel optimized)
Санах ойн төрөл	DDR4/DDR5	GDDR6

Дүн шинжилгээ: Дээрх харьцуулалтаас харахад CPU болон GPU-ийн гүйцэтгэлийн философи нь зарчмын ялгаатай:

- **CPU (Latency Oriented):** i9-12900H шиг өндөр хүчин чадалтай процессорууд нь цөөн тооны хүнд үйлдлийг маш хурдан (Low Latency) гүйцэтгэхэд зориулагдсан. Энэ нь нарийн төвөгтэй логик, салбарласан (branching) алгоритмуудыг удирдахад хамгийн тохиромжтой.
- **GPU (Throughput Oriented):** Харин RTX 3050 Ti нь нэгж ажлыг CPU-ээс хурдан гүйцэтгэдэггүй ч, мянга мянган энгийн үйлдлийг нэгэн зэрэг гүйцэтгэх чадвартай.

2.8 CUDA programming model

CUDA нь NVIDIA GPU дээр тооцоолол гүйцэтгэх параллел программчлалын платформ юм. CUDA-ийн execution model-ийн үндсэн ойлголтууд:

- **Thread** — хамгийн жижиг гүйцэтгэлийн нэгж. Нэг thread нэг дараалсан үйлдлийн дарааллыг гүйцэтгэнэ.
- **Warp** — 32 thread-ийн групп бөгөөд **SIMT (Single Instruction Multiple Threads)** зарчмаар нэг зааварыг хамт гүйцэтгэдэг. Warp дотор branch divergence гарвал гүйцэтгэл удаашрана.
- **Block (Thread Block)** — хэдэн зуу/мянган thread-ийн групп. Нэг block доторх thread-ууд **shared memory** болон synchronization ашиглан хамтран ажиллаж чадна.
- **Grid** — олон block-ийн нийлбэр. Нэг kernel дуудлага нэг grid-г launch хийдэг.
- **Streaming Multiprocessor (SM)** — GPU-ийн физик нэгж. T4 GPU-д 40 SM байх ба нэг SM 64 CUDA core-той (нийт 2,560 CUDA core).

Launch syntax:

```
kernel<<grid_dim, block_dim>>(args)
```

Манай кодонд BLOCK = 256 thread/block. 1M элементийн merge sort-ийн эхний түвшинд 500,000 thread launch хийгдэнэ.

2.9 Memory hierarchy ба host-device дамжуулалт

GPU-ийн санах ойн шатлалт нь хурдны хувьд маш ялгаатай:

Table 2: GPU memory hierarchy

Memory	Latency	Bandwidth
Register	1 cycle	—
Shared memory (SM-д)	~10 cycles	~10 TB/s
L1 cache	~30 cycles	~5 TB/s
L2 cache	~200 cycles	~2 TB/s
Global memory (GDDR6)	400-600 cycles	320 GB/s (T4)
PCIe (Host-Device)	~10 μ s	~16 GB/s (PCIe 3.0)

Үндсэн санаа: Host (CPU RAM) болон Device (GPU VRAM) хооронд өгөгдөл дамжуулах нь хамгийн удаан үе шат. Иймд GPU дээрх kernel-ийн ажил нь **дамжуулалтын зардлыг нөхөх** хэмжээнд байх ёстой.

Arithmetic intensity (AI) — нэг байт өгөгдөлд хийх үйлдлийн тоо:

$$AI = \frac{\text{Хийсэн үйлдлийн тоо}}{\text{Дамжуулсан байт}}$$

Merge sort-ын AI нь бага ($\log n$ түвшин $\times$ нэг компараци/элемент). Иймд GPU-ийн bandwidth limit-д хүрэгдэж, хурд хязгаарлагдана.

2.10 OpenMP runtime model

OpenMP нь shared-memory параллел программчлалын стандарт юм. Үндсэн загварт:

- **Fork-Join model** — `#pragma omp parallel` block-д үндсэн thread (master) олон thread fork хийдэг, block дуусахад join хийдэг.
- **Task model** — `#pragma omp task` нь recursive параллелчлалыг дэмждэг. Task scheduler нь чөлөөт thread-уудад task түгээдэг **work-stealing** стратегиар ажилладаг.
- **taskgroup** — нэг бүлэг task бүгд дуустал хүлээдэг synchronization point.

single construct нь зөвхөн нэг thread эхний task-уудыг үүсгэхийг хангадаг. Дараа нь дотор үүссэн `omp task`-уудыг үлдсэн thread-ууд work-stealing-р авч ажиллана.

2.11 Гүйцэтгэлийн үнэлгээний метрикүүд

1. Execution time: $T(n)$ — оролтын хэмжээ n -д ажиллах хугацаа. Wall-clock хугацаагаар хэмждэг.

2. Speedup:

$$S(P) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(P)}$$

P процессор ашиглавал хэдэн дахин хурдан болсон.

3. Efficiency:

$$E(P) = \frac{S(P)}{P}$$

Процессор тус бүр дунджаар хэр үр ашигтай ашиглагдсан. $E = 1.0$ нь **linear speedup**, бодит байдалд $E < 1$ байдаг.

4. Achievable performance:

$$\text{Performance} = \frac{\text{Total operations}}{\text{Execution time}}$$

Нэгж хугацаанд гүйцэтгэх үйлдлийн тоо (op/sec).

5. Strong scaling vs Weak scaling:

- **Strong scaling** — оролтын хэмжээ тогтмол байх үед процессор нэмэхэд хугацаа хэрхэн буурах
- **Weak scaling** — оролтын хэмжээ ба процессорын тоо хоёулаа нэмэгдэх үед хугацаа тогтвортой байх эсэх

Манай туршилтад **strong scaling**-ийг харьцуулсан: ижил хэмжээтэй оролт дээр өөр өөр параллел загваруудыг ашиглан хугацааг хэмжсэн.

2.12 Merge үйлдлийг параллелчлэх онолын сорилт

Хоёр sorted массивыг merge хийх энгийн алгоритмын Time Complexity нь $O(m + n)$ ба эдгээр нь **өгөгдлийн хамаарал** (data dependency)-той үйлдлүүд юм:

$$\text{output}[k] = \begin{cases} A[i] & \text{хэрэв } A[i] \leq B[j] \\ B[j] & \text{бусад тохиолдолд} \end{cases}$$

Энд i, j нь өмнөх алхмаас хамаардаг — нэг алхмын үр дүнгээс дараагийн алхамд аль массиваас элемент авах нь шийдэгдэнэ. Иймд шууд параллелчлэх боломжгүй.

Co-rank algorithm-ийн гол санаа (Siebert & Träff, 2012): гаралтын k -р индексэд харгалзах оролтын i, j индексүүдийг **бие даан** (өмнөх алхамгүйгээр) олох арга. Энэ нь thread бүр өөрийн хэсэгтэй бие даан ажиллах боломжийг олгоно:

$$\text{co_rank}(k) = i \quad \text{such that} \quad i + j = k$$

Binary search-р $O(\log(m + n))$ хугацаанд олно.

Параллел complexity:

- **Work:** $T_1 = O(n \log n)$ (sequential merge sort-той ижил)
- **Span (critical path):** $T_\infty = O(\log^2 n)$ (co-rank $\log n \times$ түвшин $\log n$)
- **Parallelism:** $\frac{T_1}{T_\infty} = \frac{n}{\log n}$

Энэ нь $n = 10^6$ үед $\sim 50,000$ дахин параллелизаци хийх боломжтой гэдгийг харуулдаг — GPU-ийн хувьд хангалттай ажил юм.

3 Туршилтын орчин

3.1 Тоног төхөөрөмж (Hardware)

Туршилтыг **nvcc** дээр гүйцэтгэсэн:

- **GPU:** NVIDIA RTX3050ti
 - Compute Capability: 7.5 (sm_75)
 - SM (Streaming Multiprocessor) тоо: 40
 - CUDA core / SM: 64

- НИЙТ CUDA CORE: 2,560
- Warp хэмжээ: 32 threads
- Max threads / block: 1024
- Max threads / SM: 1024
- Global memory: 14.6 GB GDDR6
- Memory bandwidth: 320.1 GB/s
- **CPU:** Intel I9 12900H (Colab-ийн виртуал процессор)
- **RAM:** 32GB
- **Storage:** (Хамаарлтай бус)

3.2 Програм хангамж (Software)

- **OS:** Linux Fedora 43 6.19.14-200
- **NVIDIA Driver:** 580.142
- **CUDA version:** 13.0
- **Compiler:** nvcc 13.2, g++ 15.2.1
- **OpenMP:** libgomp (GCC дотор багтсан)
- **Compile флаг:** -O2 -std=c++17 -Xcompiler -fopenmp

4 Туршилтын аргачлал

- **Туршилтын тоо:** 10 удаа давтаж ажиллуулсан (TOTAL_RUN = 10)
- **Оролтын хэмжээнүүд:** 10,000 / 100,000 / 1,000,000 элемент
- **Утгын хязгаар:** [1.0, 1000.0] float
- **std::thread / OpenMP:** 20 thread (NUM_THREADS = 20)
- **CUDA:** 256 thread/block (CUDA_THREADS_PER_BLOCK = 256)
- **CUDA warm-up:** эхний benchmark-ийн өмнө `cuda_sys.warm_up()` функцийг дуудан GPU initialization-ийг хасаж тооцсон
- **Үр дүн нэгтгэх:** 10 ажиллалтын дунджаар (mean)
- **CSV-д хадгалах талбарууд:** `method`, `input_size`, `num_threads`, `run_id`, `execution_time_ms`, `kernel_time_ms` (CUDA), `data_transfer_time`, `data_transferred_bytes`, `total_operations`, `achievable_performance`

5 Алгоритмын зохиомж

Энэ хэсэгт манай програмын код ямар бүтэцтэй бөгөөд хэрхэн ажиллаж байгааг алхам алхмаар тайлбарлав.

5.1 Ерөнхий бүтэц

Програм нь дараах файлуудаас бүрдэнэ:

- `main.cpp` — өгөгдөл үүсгэх, хугацаа хэмжих, CSV бичих
- `tasksys.cpp` — CPU алгоритмуудын хэрэгжилт
- `mergeSortCuda.cu` — CUDA kernel-ийн хэрэгжилт
- `tasksys.h` — TaskSystem class зарлал
- `itasksys.h` — интерфэйс
- `visualization/main.py` — CSV-ээс график үүсгэх Python скрипт

5.2 Өгөгдөл үүсгэх

`main` функц дотор:

1. Санамсаргүй утгатай массив үүсгэнэ (`[1.0, 1000.0]` хязгаарт)
2. `base_array`-д хадгална
3. Sort бүрийн өмнө `sorting_array` руу хуулна (`reset_array`)

5.3 Sequential хэрэгжилт

`merge_sort_serial` функц:

1. Массивыг 2 хэсэгт хуваана
2. Зүүн хэсгийг рекурсив эрэмбэлнэ
3. Баруун хэсгийг рекурсив эрэмбэлнэ
4. `merge_serial` ашиглан нэгтгэнэ

5.4 Parallel хэрэгжилт (`std::thread`)

`merge_sort_parallel` функц:

1. Хэрэв массив 10,000-аас бага бол sequential ажиллуулна (overhead-аас зайлсхийх)
2. Массивыг 2 хэсэгт хуваана
3. Зүүн хэсэгт thread үүсгэнэ
4. Баруун хэсэгт thread үүсгэнэ
5. `join()` ашиглан хүлээнэ
6. `merge_parallel_corank` ашиглан нэгтгэнэ

5.5 Co-rank ашигласан merge

`merge_parallel_corank` функц. Thread бүр дараах байдлаар ажиллана:

1. `k_start`, `k_end` утга тодорхойлно (гаралтын массивын хариуцах хэсэг)
2. `co_rank(k_start)` ашиглан эхлэх индекс олно
3. `co_rank(k_end)` ашиглан төгсгөх индекс олно
4. `merge_corank_serial` ашиглан өөрийн хэсгийг merge хийнэ

Ингэснээр thread бүр өөр өөр хэсэг дээр бие даан ажиллана.

5.6 OpenMP хэрэгжилт

`merge_sort_parallel_openmp` функц нь `#pragma omp task` ашиглан рекурсив параллелчлал хийнэ:

- `#pragma omp parallel num_threads(20)` — 20 thread-ийг fork
- `#pragma omp single` — зөвхөн нэг thread эхний task-уудыг үүсгэнэ
- `#pragma omp task` — recursive call бүрийг task болгоно
- `#pragma omp taskgroup` — merge хийхээс өмнө хоёр хагасыг хүлээнэ

5.7 CUDA хэрэгжилт

CUDA код нь `TaskSystemCUDA` class дотор:

- **Memory reuse:** `device_input_`, `device_temp_` буферыг давтан ашиглана (`cudaMalloc`-ийг олон удаа дуудахаас зайлсхийх)
- **warm_up():** эхний дуудлагын өмнө GPU-г initialize-лах
- **Bottom-up merge sort:** `width = 1, 2, 4, ..., n/2` түвшинд явна
- **Block per pair:** kernel-ийн grid нь pair тоотой тэнцүү, нэг block нэг merge pair-г хариуцна
- **Threads cooperate:** block доторх 256 thread co-rank-р хуваан merge хийнэ

`merge_pass_kernel`-ийн алхам:

1. Block ID-аас pair-ийн эхлэлийн индексийг тооцоолно
2. Pair-ийн зүүн ба баруун хэсгийг тодорхойлно (boundary clamp хийнэ)
3. Thread бүр өөрийн `k_start`, `k_end`-г олно
4. `co_rank`-р оролтын индексүүдийг тооцоолно
5. Sequential merge хийгээд гаралтын массивт бичнэ

5.8 Нийт ажиллагаа

1. Оролтын өгөгдөл үүсгэнэ
2. CUDA warm-up хийнэ (анхдагч initialization-ийг хасах)
3. Run loop (10 удаа):
 - Size loop (10K, 100K, 1M):
 - Serial хувилбарыг ажиллуулна
 - std::thread хувилбарыг ажиллуулна
 - OpenMP хувилбарыг ажиллуулна
 - CUDA хувилбарыг ажиллуулна
 - Үр дүн зөв эсэхийг шалгана (checker)
 - Хугацаа болон performance-г CSV файлд хадгална
4. Python ашиглан график байгуулна

6 Үр дүн ба харьцуулсан шинжилгээ

Туршилтыг 10,000, 100,000, 1,000,000 хэмжээтэй өгөгдөл дээр хийж, алгоритм бүрийг 10 удаа ажиллуулан дундаж үр дүнг гаргасан. Үр дүнг CSV файлд хадгалж Python ашиглан график байгуулсан.

6.1 Тоон үр дүн

Table 3: Дундаж execution time (мс) ба Speedup (10 ажиллалтын дундаж)

Method	N=10K		N=100K		N=1M	
	ms	SU	ms	SU	ms	SU
Serial	0.66	1.00×	7.97	1.00×	94.31	1.00×
std::thread	0.65	1.03×	2.93	2.72×	12.09	7.80×
OpenMP	0.80	0.82×	1.02	7.78×	11.52	8.19×
CUDA	0.32	2.06×	2.36	3.37×	24.03	3.93×

6.2 CUDA-ийн хугацааны задлал

CUDA хугацаа = H2D дамжуулалт + Kernel ажиллах + D2H дамжуулалт. Манай туршилтад дараах байдалтай:

Table 4: CUDA-ийн хугацаа задлал (10 ажиллалтын дундаж)

N	Total (ms)	Kernel (ms)	Transfer (ms)	Bytes
10,000	0.32	0.26	0.03	80 KB
100,000	2.36	2.22	0.14	800 KB
1,000,000	24.03	23.24	0.95	8 MB

Эндээс харахад дамжуулалт нь нийт хугацааны ~4-9% эзэлдэг бол kernel-ийн ажиллах хугацаа ~91-96% хувийг эзэлж байна. Энэ нь GPU-ийн ачаалал хангалттай биш гэдгийг харуулна.

6.3 Execution Time

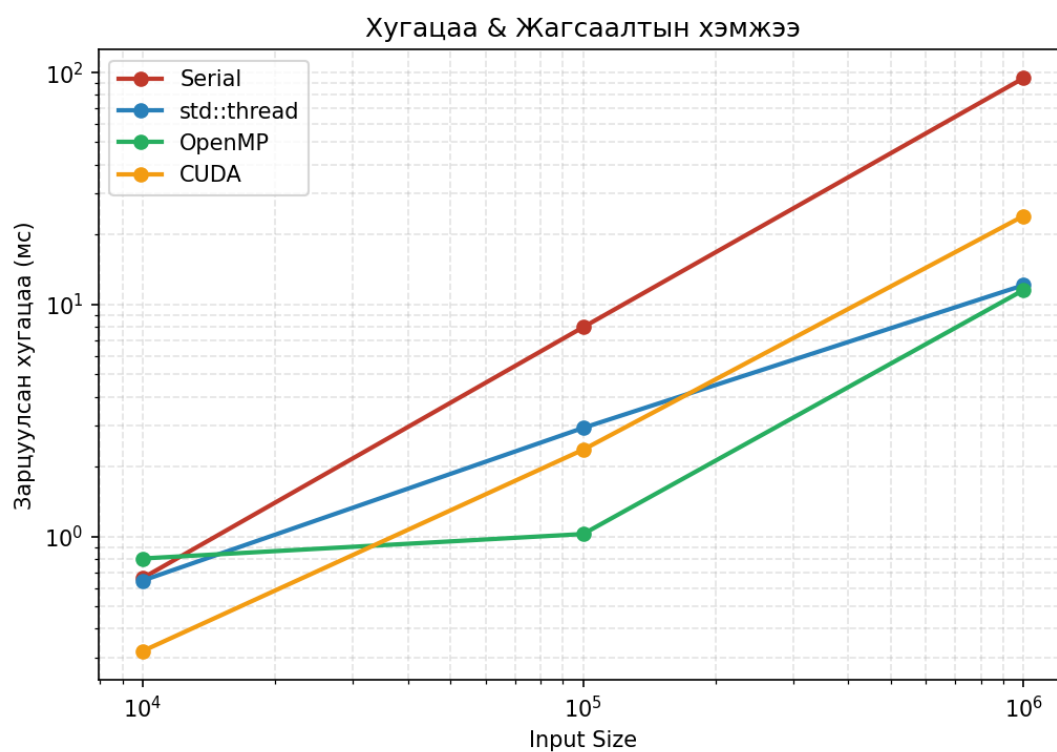


График: Execution time харьцуулалт (log-log scale)

Графикаас харахад оролтын хэмжээ өсөх тусам бүх алгоритмын хугацаа өссөн байна. Serial хувилбар хамгийн удаан ажиллаж байгаа бол OpenMP болон std::thread хувилбарууд илүү хурдан байна.

CUDA хувилбар нь жижиг хэмжээтэй өгөгдөл (10K) дээр хамгийн хурдан байсан боловч өгөгдлийн хэмжээ ихсэх үед OpenMP-той харьцуулахад удаан болсон байна. Энэ нь манай CUDA implementation-ийн хязгаарлалттай холбоотой (доор тайлбарласан).

6.4 Speedup

$$\text{Speedup} = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

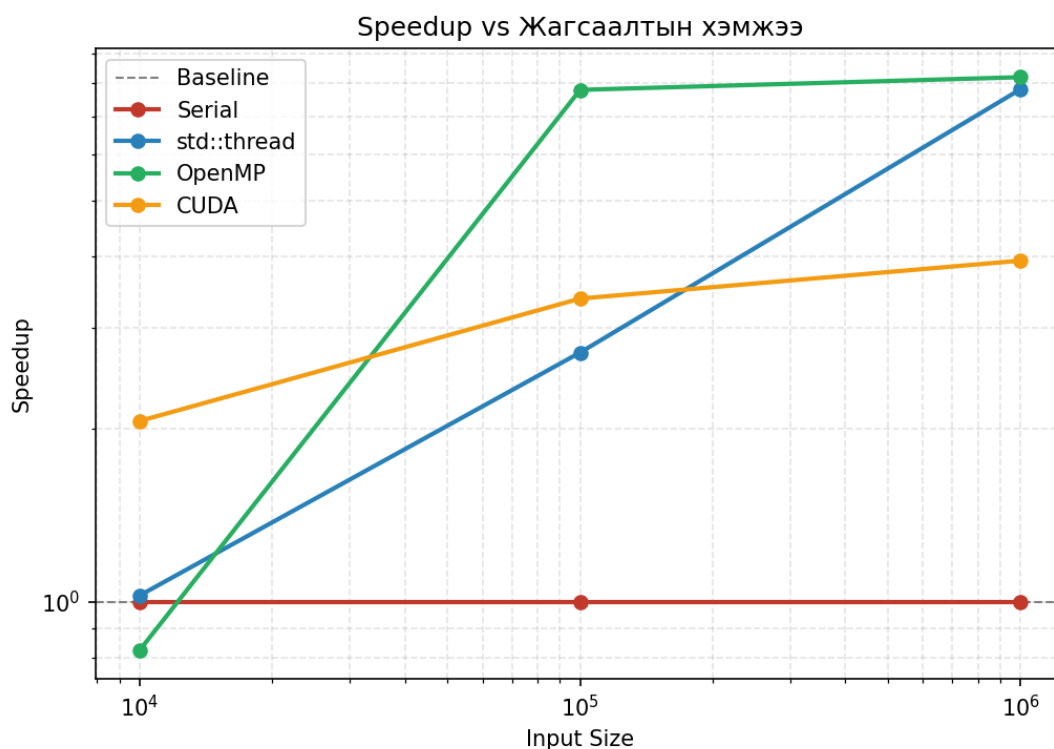


График: Speedup харьцуулалт (log-log scale)

Speedup графикаас харахад өгөгдлийн хэмжээ өсөх тусам параллел хувилбаруудын давуу тал нэмэгдэж байна.

OpenMP хувилбар хамгийн өндөр speedup ($8.19\times$) үзүүлж байгаа нь CPU-ийн олон цөмийг үр ашигтай ашиглаж байгааг харуулж байна. std::thread хувилбар мөн сайн үр дүн ($7.80\times$) үзүүлсэн.

CUDA хувилбар нь 1M элементэд $3.93\times$ хурдатгал үзүүлсэн боловч OpenMP-оос бага speedup үзүүлсэн байна. Энэ нь дараах хэсэгт тайлбарлагдсан CUDA-ийн implementation-ийн өвөрмөц хязгаарлалттай холбоотой.

6.5 Achievable Performance

$$\text{Performance} = \frac{\text{total_operations}}{\text{execution_time}}$$

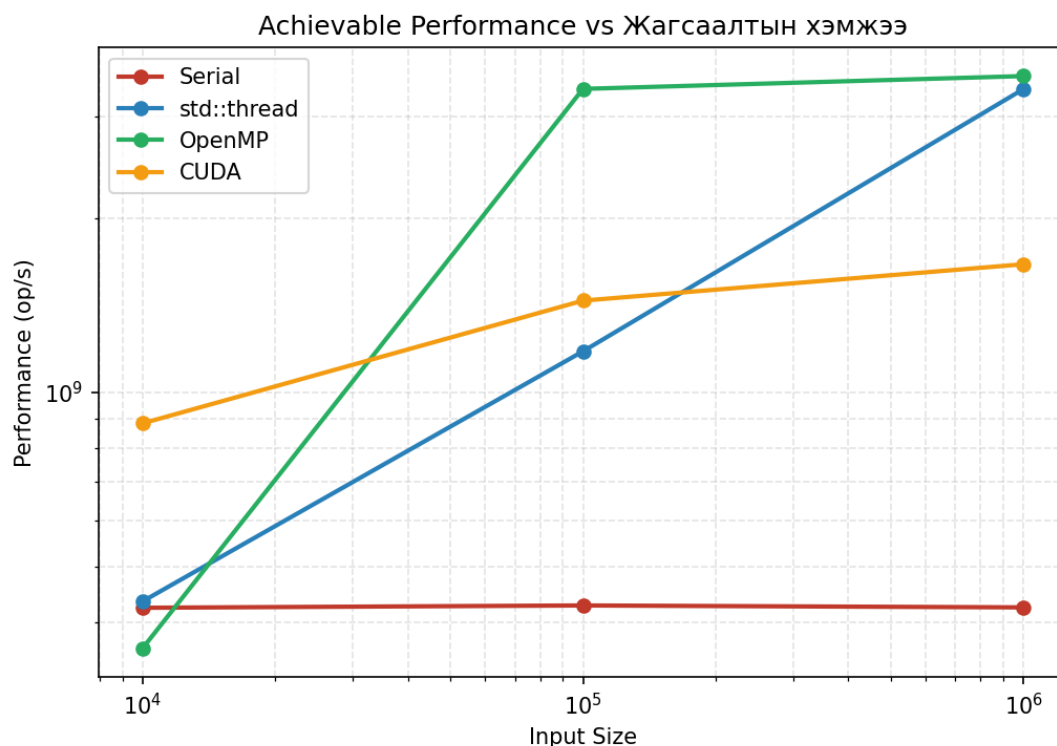


График: Achievable performance харьцуулалт (op/sec)

Achievable performance графикаас харахад OpenMP болон std::thread хувилбарууд илүү өндөр гүйцэтгэл үзүүлж байна.

CUDA хувилбар нь жижиг өгөгдөл дээр өндөр performance үзүүлсэн боловч CPU дээрх OpenMP хувилбараас 1M-д бага үзүүлэлттэй байна.

Serial хувилбарын гүйцэтгэл хамгийн бага байгаа нь нэг thread дээр ажиллаж байгаатай холбоотой.

6.6 CUDA implementation-ийн хязгаарлалт ба тайлбар

Манай CUDA kernel-ийн загвар нь нэг merge pair = нэг CUDA block зарчмаар ажилладаг (blockIdx.x = pair_index). Энэ нь дараах онцлогтой:

- **Жижиг хэмжээнд (10K) илүү:** олон pair → олон block → GPU бүрэн ачаалагдана. Жишээ нь 10K-ийн эхний түвшинд 5,000 block үүсгэнэ.
- **Том хэмжээнд (1M) хязгаарлалт:** эцсийн merge-т зөвхөн 1 block (256 thread) ажиллана. Tesla T4-ийн 2,560 CUDA core-ийн дөнгөж ~10%-г ашигладаг.

Үе шат бүрд launch болох block-ийн тоо (1M элементэд):

Width	Pair тоо	Тайлбар
1	500,000	GPU бүрэн ачаалагдана (~ 1953 SM-ийн ачаалал)
256	1,953	
1,024	488	GPU сайн ачаалагдана
524,288	2	Дунд зэргийн ачаалал
		Зөвхөн 2 block — GPU-ийн дөнгөж ~ 5%

OpenMP-д $20 \text{ thread} \times \text{CPU-ийн SIMD execution}$ = эффектив илүү parallelism гардаг тул CUDA-аас илүү хурдан болж байна.

Боломжит сайжруулалт: *Co-rank-ийг олон block хооронд тарааж*, нэг pair-ийг олон block-р хамтран merge хийх hybrid kernel ашиглах. Гэхдээ энэ ажилд ердийн (single-block-per-pair) загвар нь ойлгомжтой бөгөөд co-rank-ийн үндсэн зарчмыг харуулах зорилгод хангалттай.

6.7 Ерөнхий ажиглалт

Жижиг хэмжээтэй өгөгдөл дээр параллел алгоритмын давуу тал бага байсан. Харин өгөгдлийн хэмжээ өсөх тусам `std::thread` болон OpenMP хувилбаруудын гүйцэтгэл мэдэгдэхүйц сайжирсан.

Co-rank алгоритм ашигласнаар merge үйлдлийг thread бүрт хуваарилж, параллел гүйцэтгэлийг нэмэгдүүлэх боломж бүрдсэн.

CUDA хувилбар нь GPU ашигласан боловч өгөгдөл дамжуулах болон kernel ажиллуулах нэмэлт хугацаа, мөн нэг pair = нэг block загварын хязгаарлалтаас шалтгаалан 100K, 1M-ийн хувьд CPU дээрх OpenMP хувилбараас бага гүйцэтгэл үзүүлсэн.

7 Дүгнэлт

Энэхүү бие даалтын ажлаар Merge Sort алгоритмыг serial, `std::thread`, OpenMP болон CUDA хувилбаруудаар хэрэгжүүлж, гүйцэтгэлийг харьцуулсан.

Execution time графикаас харахад оролтын хэмжээ өсөх тусам бүх алгоритмын хугацаа нэмэгдэж байгаа боловч параллел хувилбарууд илүү хурдан ажилласан. Ялангуяа OpenMP болон `std::thread` хувилбарууд serial хувилбараас илүү сайн гүйцэтгэл үзүүлсэн.

Speedup графикаас харахад өгөгдлийн хэмжээ ихсэх тусам параллел алгоритмуудын давуу тал нэмэгдэж байгаа нь ажиглагдсан. OpenMP хувилбар хамгийн өндөр speedup ($8.19\times$) үзүүлсэн бол `std::thread` хувилбар ($7.80\times$) мөн тогтвортой өсөлттэй байсан.

Achievable performance графикаас харахад OpenMP болон `std::thread` хувилбарууд илүү өндөр гүйцэтгэлтэй ажилласан. Харин serial хувилбар хамгийн бага гүйцэтгэлтэй байсан.

CUDA хувилбарын хувьд GPU ашигласан боловч execution time болон performance графикаас харахад OpenMP-той харьцуулахад бага үзүүлэлттэй байсан. Энэ нь:

- Host-Device дамжуулалтын тогтмол overhead
- Манай implementation-ийн “нэг pair = нэг block” хязгаарлалт (том width-д GPU дутуу ачаалагдах)
- Merge sort-ын arithmetic intensity бага байх

шалтгаануудтай холбоотой.

Гэхдээ CUDA хувилбар жижиг хэмжээний өгөгдөл (10K) дээр $2.06\times$ speedup үзүүлсэн нь GPU нь зохистой ачаалал бүхий жижиг ажиллагаанд тохиромжтой гэдгийг харуулсан.

Ерөнхийдөө өгөгдлийн хэмжээ өсөх тусам параллел алгоритмуудын үр ашиг нэмэгдэж, OpenMP хувилбар хамгийн тогтвортой, өндөр гүйцэтгэл үзүүлсэн. Туршилтаас гарсан гол сургамж бол:

1. Параллел загварыг сонгохдоо алгоритмын онцлог, өгөгдлийн хэмжээг харгалзан үзэх ёстой.
2. GPU нь массив параллелиزم бүхий, arithmetic intensity өндөр алгоритмд тохиромжтой.
3. CPU multi-threading (OpenMP) нь дунд хэмжээний $\log n$ алгоритмуудад үр ашигтай.
4. Co-rank algorithm нь параллел merge-ийг correct, stable байдлаар хэрэгжүүлэх онолын суурь юм.

8 Эх сурвалж

References

- [1] Kirk, D. B., & Hwu, W. M. W. (2016). *Programming Massively Parallel Processors: A Hands-on Approach* (3rd ed.). Morgan Kaufmann. Chapter 12: Merge.
- [2] Siebert, C., & Träff, J. L. (2012). A Correct and Stable Parallel Merge Algorithm. *Information Processing Letters*, 113(11).
- [3] NVIDIA Corporation. (2024). *CUDA C++ Programming Guide v13.0*.
- [4] OpenMP Architecture Review Board. (2021). *OpenMP API 5.2 Specification*.
- [5] Amdahl, G. M. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. *AFIPS Conference Proceedings*, 30, 483–485.
- [6] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Chapter 27: Multithreaded Algorithms.
- [7] Г. Ганбат. (2026). Параллел ба хуваарилагдсан тооцоолол — Лекцийн материал. F.CSM306, ШУТИС МХТС.